CANDY
DIALOGUE ENGINE

UI CUSTOMIZATION

1.1.0

The following principles apply when customizing any UI Elements:

Basics

- UI Elements should be saved as standalone scenes in your resource folders.
- UI Element scenes can be dynamically loaded/removed at runtime, if you want to swap between designs or layouts at various points.
- The node structure of any UI Element can be completely customized, except for a few Required Nodes that must be present.
- The size, location, appearance or other properties of UI Elements can be fully customized.
- Most UI Elements require a script, which requires some variables and functions depending on the type of element.
- UI Element scripts can often be heavily customized, with most functions acting as hooks.
- Basic scripts are provided for each type of element, and can be used as-is or as templates to be modified or expanded.

In simple words:

1) Create UI Elements however you like.
2) Make sure they feature the few nodes Candy DE requires.
3) Attach the script that matches the element type to the scene's root node (e.g. dialogue box script for a custom dialogue box)
4) Optionally, modify the script to customize the element's behavior.

Node Structure

1) As mentioned, there aren't many rules when it comes to node structure: any Required Nodes must be present.

For example, a dialogue box needs a two nodes capable of displaying text: one for the spoken text, and one for the speaker's name. Either of the following node structures would be valid:

- Control (root node)
- - RichTextLabel (speaker node) [REQUIRED]
- - RichTextLabel (spoken text node) [REQUIRED]

- PanelContainer (root node) [REQUIRES SCRIPT]
- - VBoxContainer
- - - RichTextLabel (speaker node) [REQUIRED NODE]
- - - ScrollContainer
- - - - RichTextLabel (spoken text node) [REQUIRED NODE]

...or any other structure would work.

2) The required nodes must be exported in the UI Element's script, like this:

```
@export var dialogue_node: Node
@export var speaker_node: Node
```

The exact nodes can then be assigned in the Inspector.

This allows the dialogue engine to find the nodes it requires, so that it can use them.

3) In rares cases, some Required Nodes require specific parents or specific children.

For example, Background Scenes require nodes capable of displaying background layers (e.g. TextureRect). These layer nodes must all be children of the same parent node, and that parent node requires a pointer in the script.

All details are provided in the Customization guide for each UI Element.

Script

As mentioned, UI Elements require a script attached to their root node. That script must contain the following:

Required Node Pointers

As explained previously, a UI Element script must contain pointers to Required Nodes that the dialogue refers to.

Variables

UI Element scripts usually contains various other variables, which serve different purposes depending on the type of UI Element.

Caller

A commonly shared variable is '**caller**'. This variable indicates which instance of the dialogue engine is using the UI Element. This is relevant in case your game runs multiple instances of the dialogue engine.

Default_Z

Another variable found in all UI Element scripts is **default_z**. It determines the z_index value you want to use by default for the given UI Element, and is customizable (it's an exported variable, therefore it can be modified in the Inspector).

For beginners, the z_index of a node is essentially its layer position. Nodes with a higher z_index appear on top of nodes with a lower z_index.

Candy DE sometimes alters the z_index of some UI Elements, for example to overlay the dialogue box above video or image media, or conversely to hide it under such media. When this happens, Candy DE will use **default_z** to return a UI Element to your desired default z_index.

Functions

UI Element scripts may also contain functions that are called upon by the dialogue engine, or which are otherwise required for a given UI Element to perform correctly.

All functions can be modified to fit your needs and preferences, however, unless explicitly encouraged, this should only be done if you are confident you understand the code.

Ready

The **_ready()** function in UI Element scripts sometimes contains code. Like functions, such code shouldn't be modified unless you know what you're doing or you are encouraged to customize it.